

Section A: Introduction to Docker (10 Questions)

1. What is Docker, and how does it differ from a virtual machine?

Docker is a containerization tool that packages apps with dependencies. Unlike VMs, it shares the host OS kernel, so it's lightweight and faster.

2. Write a basic Dockerfile for a Flask application.

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

3. Explain the role of **Docker images** vs **Docker containers**.

- Image = blueprint (like a class).
- Container = running instance of the image (like an object).

4. How do you list all running Docker containers and stop one?

```
docker ps
docker stop <container_id>
```

5. What is Docker Hub, and how is it used in deployments?

It's a cloud registry to store and share Docker images. Used to pull/push images during deployments.

6. Debug: Your container exits immediately after running. What could be wrong?

The main process in CMD/ENTRYPOINT finished, so the container stopped.

7. Explain the difference between docker build and docker run.

- docker build: Creates an image from a Dockerfile.
- docker run: Starts a container from an image.

8. What is the purpose of .dockerignore file?

It excludes files/folders (like .git, venv) from being copied into the image → reduces size.

9. How do you expose a Flask app running inside Docker on port 5000?

```
docker run -p 5000:5000 flask-app
```

10. Compare Docker with Kubernetes briefly.

- Docker: Creates and runs containers.
- Kubernetes: Orchestrates and manages multiple containers at scale.

Section B: Containerization of Microservices (15 Questions)

11. Why is Docker especially useful in microservices architecture?

Each service can run in its own container with dependencies isolated, making scaling and deployment easier.

12. Write a docker-compose.yml file for two services: Flask API and MongoDB.

```
version: '3'

services:
  api:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - mongo
  mongo:
    image: mongo:latest
    volumes:
      - mongo-data:/data/db
volumes:
  mongo-data:
```

13. Explain the difference between Docker volume and bind mount.

- Volume: Managed by Docker, stored in `/var/lib/docker`.
- Bind mount: Maps host directory directly to container.

14. How do you persist database data in Dockerized microservices?

By using volumes so data is not lost when the container restarts.

15. Debug: Flask container cannot connect to MongoDB container. What should you check?

- Check Docker network setup.
- Use service name (`mongo`) instead of `localhost`.
- Check exposed ports.

16. What is the role of **Docker networks** in microservices communication?

They allow containers to communicate securely by name instead of IP.

17. How do you scale a Flask service to 3 replicas using Docker Compose?

```
docker-compose up --scale api=3
```

18. Write the command to build and run a container with name `school-api` from a Dockerfile.

```
docker build -t school-api .
```

```
docker run -d --name school-api -p 5000:5000 school-api
```

19. Compare `docker-compose` and Kubernetes for multi-service deployments.

- `docker-compose`: Simple, local, for small projects.
- Kubernetes: Advanced, scalable, production-ready orchestration.

20. How do you pass environment variables securely to containers?

Use `.env` files with `docker-compose`, or Docker secrets for sensitive data.

21. Explain the use of **multi-stage builds** in Dockerfiles.

They reduce image size by separating build and runtime stages, copying only the needed artifacts to the final image.

22. How would you optimize a Docker image for Python apps?

- Use slim base images.
- Add `.dockerignore`.
- Install only required dependencies.
- Use multi-stage builds.

23. What's the impact of containerization on CI/CD pipelines?

It ensures consistency across environments, faster testing, and easy deployment.

24. Describe how you would containerize a Django + PostgreSQL project.

- Write a Dockerfile for Django.
- Use docker-compose to link Django service with Postgres service.
- Add volumes for DB persistence.

25. What is Docker Swarm, and how is it different from Kubernetes?

Docker Swarm is Docker's native clustering/orchestration tool. Kubernetes is more powerful, widely used, and has richer features.

Section C: Consuming REST API in SPA (15 Questions)

26. What is a Single Page Application (SPA)? Give 2 examples.

An SPA loads once and updates dynamically using APIs. Examples: Gmail, Facebook.

27. How do SPAs typically consume REST APIs?

They use AJAX/Fetch/Axios to call APIs asynchronously and update the UI without reload.

28. Write JavaScript code using **fetch API** to call `/api/students` and display results.

```
fetch('/api/students')  
  .then(res => res.json())  
  .then(data => console.log(data));
```

29. What are CORS issues, and how do you fix them in Flask/Django backend?

CORS blocks requests from different origins. Fix by enabling CORS with libraries like flask-cors or Django's django-cors-headers.

30. Debug: Your React app gets Network Error when calling Flask API at `http://localhost:5000`. Why?

Because frontend runs on port 3000 and backend on 5000 → CORS issue or wrong base URL.

31. How do you handle JWT tokens in SPA for authentication?

Store token in localStorage or cookies, attach it in Authorization headers for API requests.

32. Write React code to make a POST request to `/api/login`.

```
fetch('/api/login', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ username, password })  
})  
.then(res => res.json())  
.then(data => console.log(data));
```

33. What is the role of Axios in consuming REST APIs?

Axios is a promise-based HTTP client that simplifies API calls with cleaner syntax and built-in error handling.

34. Compare server-side rendering (SSR) vs SPA.

- SSR: Server renders HTML (faster initial load, SEO-friendly).
- SPA: Browser renders via JS (dynamic, smoother navigation).

35. How do you handle loading states in a SPA when fetching API data?

Use state variables (e.g., `isLoading`) to show a spinner or message until data arrives.

36. Write Vue.js code to consume a Flask API endpoint.

```
axios.get('/api/data')  
  .then(res => { this.items = res.data })  
  .catch(err => console.error(err));
```

37. How do you handle API errors gracefully in a SPA?

Catch errors, show user-friendly messages, maybe retry or redirect.

38. Explain pagination in API responses. How does an SPA handle it?

API sends limited results with page and limit. SPA handles it by fetching next page when user navigates or scrolls.

39. What are the security risks of exposing APIs directly to SPAs?

Risks include token theft, brute-force attacks, and exposing sensitive endpoints. Must use authentication, rate limiting, and HTTPS.

40. Design an example where a **student dashboard SPA** consumes microservices APIs for attendance, grades, and library data.

The SPA frontend makes API calls like:

- /api/attendance → Attendance microservice
- /api/grades → Grades microservice
- /api/library → Library microservice

Then aggregates results to display on the dashboard.

Microservices Design:

1. Attendance Service:

- Manages attendance records per student.
- API endpoint: GET /attendance/{student_id} → returns days present, absent, and percentage.

2. Grades Service:

- Manages exam scores and subject grades.
- API endpoint: GET /grades/{student_id} → returns list of subjects and scores.

3. Library Service:

- Tracks borrowed books, due dates, and fines.
- API endpoint: GET /library/{student_id} → returns list of borrowed books and fines.